Pipeline ELT: Análisis de Accidentes de Tráfico en Brasil

Introducción a la Integración de Datos — MIA 03

Abril 2026

Sergio Morel, Clara Almirón & Daniel Ramírez

Contents

1 Introducción 4
1.a Contexto y objetivo 5
1.b Arquitectura general 6
2 Fuentes de datos 7
2.a Fuente 1 — DATATRAN PRF 2026 8
2.b Fuente 2 — Open-Meteo ERA5 9
3 Pre-procesamiento y carga 10
3.a Paso 1 — Conversión del CSV 11
3.b Paso 2 — MySQL con Docker 12
4 Airbyte 13
4.a Configuración de la conexión 14
5 dbt — Transformaciones 15
5.a Lineage graph 16
5.b Capa staging 17
5.c Capa intermedia y OBT 18
5.d Tests de calidad 19

Contents (ii)

6 Prefect — Orquestación 20
6.a Pipeline datatran_pipeline 21
6.b Resultado de ejecuciones 22
7 Metabase — Dashboard 23
7.a Dashboard interactivo 24
7.b Hallazgos principales 25
8 Conclusiones 26
8.a Decisiones técnicas clave 27
8.b Resumen del pipeline 28

1 Introducción

Contexto y objetivo

Problemática: los accidentes de tráfico son una de las principales causas de mortalidad en Brasil. La PRF registra cada accidente en rutas federales, pero la condición climática es anotada subjetivamente por el agente.

Objetivo: enriquecer el dataset de accidentes con datos climáticos objetivos (ERA5) para analizar la relación entre clima y accidentalidad.

Resultado: pipeline ELT completo con dos fuentes integradas, modelo analítico en MotherDuck y dashboard interactivo en Metabase.

Arquitectura general

Extracción	Carga	Warehouse	Transformación	Visualización
MySQL (Docker)	Airbyte	MotherDuck	dbt	Metabase
Open-Meteo ERA5	↑ Prefect		15/15 tests	Dashboard + filtros

Orquestación: Prefect ejecuta las 7 etapas de extremo a extremo con un único comando.

2 Fuentes de datos

Fuente 1 — DATATRAN PRF 2026

- **Origen:** Policía Rodoviária Federal — dados.gov.br
- **Contenido:** 11.380 accidentes en rutas federales, enero–abril 2026
- **Formato:** CSV con separador `;`, encoding Latin-1, fin de línea CRLF
- **30 columnas:** localización (UF, municipio, BR, km, lat/lon), causa, tipo, fase del día, condición meteorológica PRF, víctimas, vehículos

Desafíos de ingesta:

- Decimales con coma: `-7,291548` ⟶ requiere `REPLACE` antes de castear
- Campo `tracado_via` con `;` interno ⟶ debe respetar entrecomillado CSV
- Valores nulos literales: `NA` ⟶ `NULLIF` en dbt
- Encoding Latin-1 ⟶ conversión a UTF-8 antes de cargar en MySQL

Fuente 2 — Open-Meteo ERA5

- **Origen:** archive-api.open-meteo.com — reanalysis ERA5, resolución horaria
- **Costo:** gratuito, sin API key
- **Join con DATATRAN:** `(ROUND(lat,2), ROUND(lon,2), fecha, hora)`

Variable ERA5	Relevancia vial
`precipitation`	★★★ Precipitación mm/h
`weather_code` (WMO)	★★★ Condición estandarizada
`temperature_2m`	★★ Temperatura ambiente
`cloud_cover_low`	★ Nubes bajas — proxy niebla
`wind_gusts_10m`	★ Ráfagas de viento
`shortwave_radiation`	★ Radiación solar — encandilamiento

3 Pre-procesamiento y carga

Paso 1 — Conversión del CSV

`workspaces/scripts/convert_csv_to_utf8.py`

- Lee los `.csv` de `CSV_SOURCE_DIR` (configurable en `.env`)
- Usa el módulo `csv` de Python con `quotechar='"'` para respetar `tracado_via`
- Escribe archivos `*_utf8.csv` en `data/` (UTF-8, LF)

```
python3 workspaces/scripts/convert_csv_to_utf8.py
# Filas escritas (incluye encabezado): 11381
```

Paso 2 — MySQL con Docker

```yaml
services:
  tf-mysql:        # puerto 3306
  tf-phpmyadmin:   # puerto 8095
  tf-metabase:     # puerto 3000
```

Decisión clave — todo TEXT en la capa raw:

Campo	Problema
`latitude`, `longitude`, `km`	Decimal con coma
`classificacao_acidente`	Valor literal `NA`
`data_inversa`, `horario`	Strings — dbt castea a DATE/TIME

La lógica de tipado queda 100% en dbt: versionable, testeable, auditable.

4 Airbyte

Configuración de la conexión

Source: `MySQL_Datatran` — base `datatran`, usuario `airbyte`, Full Refresh

Destination: `MotherDuck_datatran` — `md:airbyte_trabajo`, schema `datatran`

Connection: `MySQL_Datatran → MotherDuck_datatran`

- Schedule: **Manual** (Prefect dispara cada ejecución)
- Sync mode: **Full Refresh | Overwrite** (dataset estático, sin timestamp de cambio)

Dos streams sincronizados:

Tabla	Campos	Filas
`accidentes_raw`	30	11.380
`clima_raw`	13	53.000

5 dbt — Transformaciones

Lineage graph

El grafo de dependencias muestra el flujo completo:

`datatran.accidentes_raw` `datatran.clima_raw`

↓ ↓

`stg_accidentes` `stg_clima`

↘ ↙

`int_accidentes_clima`

↓

`obt_accidentes` *(marts — materializado como tabla)*

Capa staging

`stg_accidentes`

- `NULLIF(campo, 'NA')` para nulos literales
- `REPLACE(latitude, ',', '.')::DOUBLE` — decimales con coma
- `id_accidente = row_number()` — surrogate key propio (no confiar en el id DATATRAN)
- `lat_r = ROUND(latitude, 2)`, `lon_r = ROUND(longitude, 2)` — clave de join

`stg_clima`

- Deduplicación por `row_number()` sobre `(latitude, longitude, timestamp)`
- Decodificación del código WMO a descripción legible (`weather_desc`)
- Columnas de join: `lat_r`, `lon_r`, `fecha`, `hora`

Capa intermedia y OBT

`int_accidentes_clima`

```
LEFT JOIN stg_clima c
  ON  a.lat_r         = c.lat_r
  AND a.lon_r         = c.lon_r
  AND a.data_inversa  = c.fecha
  AND hour(a.horario) = c.hora
```

`clima_join_match = TRUE` cuando se encontró dato ERA5 para el accidente.

`obt_accidentes` — One Big Table (57 columnas):

- 11.380 filas, materializada como `table` en MotherDuck
- Razón para OBT: volumen pequeño, análisis exploratorio, DuckDB no penaliza tablas anchas

Tests de calidad

Test	Modelo	Dimensión
`unique` + `not_null`	`id_accidente`	Unicidad / completitud
`expect_column_values_to_be_between`	`mortos` 0–100	Validez de rango
`expect_column_values_to_be_between`	`latitude` −34 a 6	Validez geográfica
`not_null`	`uf`	Completitud
`expect_column_values_to_be_between`	`data_inversa`	Validez temporal
`expect_column_proportion_of_unique_values`	`municipio`	Consistencia

Resultado: 15/15 tests pasan, 0 warnings.

6 Prefect — Orquestación

Pipeline datatran_pipeline

`workspaces/prefect/pipeline.py` — 7 tasks ejecutadas en secuencia:

Task	Descripción
`ensure_containers_up`	`docker compose up -d` + espera MySQL (máx 60s)
`load_accidentes_raw`	Convierte CSVs ⟶ recrea tabla ⟶ LOAD DATA LOCAL INFILE
`extract_openmeteo`	Extrae ERA5 por fecha/coordenada, reanudación via `.progress`
`load_clima_raw`	Crea `clima_raw` y carga CSV (saltea si ya tiene datos)
`airbyte_sync`	Dispara sync via API REST, polling cada 10 s
`dbt_run`	`dbt deps` + `dbt run`
`dbt_test`	`dbt test` — falla el flow si algún test no pasa

Resultado de ejecuciones

- **9 ejecuciones** registradas durante el desarrollo
- Las 8 ejecuciones fallidas corresponden a iteraciones de corrección: autenticación Airbyte, rutas dbt, deduplicación de clima
- **Última ejecución:** ✅ 7/7 tasks completadas, 15/15 tests pasan

```
cd workspaces/prefect
python pipeline.py
# UI disponible en http://localhost:4200
```

7 Metabase — Dashboard

Dashboard interactivo

5 paneles sobre `marts.obt_accidentes`, con dos filtros propagados a todos:

- **Estado** — selector de UF
- **Fechas** — rango sobre `data_inversa`

#	Tipo	Pregunta
1	Dona	¿Cuáles son las 5 principales causas de accidentes?
2	Barras	¿Cómo se distribuyen los accidentes por día de la semana?
3	Líneas doble eje	¿Cómo evolucionan accidentes y muertes en la semana?
4	Dona	¿Qué condición climática concentra más accidentes?
5	Barras agrupadas	¿Qué condición climática genera más muertes y heridos?

Hallazgos principales

Causas: *Ausência de reação* (30%) y *Reação tardia* (29%) acumulan 60% — la atención del conductor es el factor dominante.

Día de la semana: los accidentes de fin de semana tienen mayor gravedad relativa (curva de muertes crece más que la de accidentes a partir del viernes).

Clima: *Céu Claro* concentra 55% de los accidentes — el clima no es el factor principal. Sin embargo, *Vento* y *Chuva* presentan las mayores proporciones de personas afectadas por accidente.

Filtro por RJ: la distribución de causas se mantiene estable; *Chuva* tiene mayor peso relativo (19.6% vs 14.3% nacional) — consistente con el clima de Río de Janeiro.

8 Conclusiones

Decisiones técnicas clave

- **Todo TEXT en MySQL raw:** evita fallos de ingesta; el tipado queda íntegramente en dbt donde es testeable y versionable.
- **Open-Meteo ERA5 sobre OpenWeather:** gratuito, sin API key, más variables útiles (ráfagas, nubes bajas, radiación solar).
- **OBT en lugar de estrella Kimball:** con 11.380 filas el volumen no justifica normalización. DuckDB/MotherDuck no penaliza tablas anchas y Metabase se simplifica.
- **Surrogate key propio (`id_accidente`):** el id de DATATRAN no es confiable como clave única — generamos nuestro propio `row_number()` en dbt.
- **Prefect orquesta todo:** un solo `python pipeline.py` ejecuta desde la conversión del CSV hasta `dbt test`, garantizando reproducibilidad.

Resumen del pipeline

Componente	Estado	Resultado
Extracción (Airbyte)	✅	2 tablas sincronizadas en MotherDuck
Transformación (dbt)	✅	3 capas, OBT con 57 columnas
Calidad (dbt-expectations)	✅	15/15 tests pasan
Orquestación (Prefect)	✅	7 tasks, ejecución end-to-end
Visualización (Metabase)	✅	5 paneles + filtros interactivos
Reporte técnico	✅	Typst, incluye capturas y análisis